WEEK 09

Functions, filters and tests

This week: learn, practise, build

01 · Python

Functions, filters and tests
Understand the idea and trace examples.

02 · Practice

Practise the same Python concept.
Predict, change, diagnose and explain.

03 · Project

Student-designed feature
Build, discuss and test with a peer.

One weekly component moves the final project forward.

Python foundations

Functions, filters and tests
Concept → trace a small example → explain its behavior

Predicates, result lists and independent tests

Explicit question

Select enabled tasks that are unfinished.

Predicate and loop

Append an ID only when both required conditions hold.

Independent tests

Finished and disabled examples distinguish wrong filters.

Choose test inputs that distinguish the intended predicate from a wrong one.

Specify behavior before changing a function

Function contract

Name the inputs and expected returned value. Keep an ordinary case beside a boundary case.

Filter by a predicate

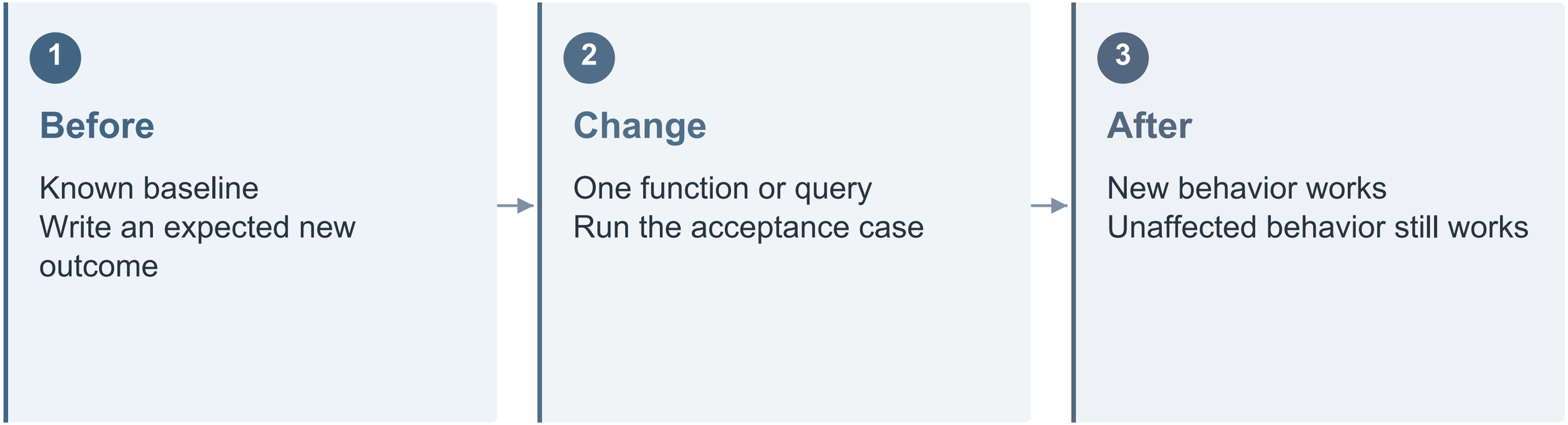
A filter keeps records whose condition is True. State exactly which records should remain.

Assertion as evidence

assert compares an observation with an expectation. Also rerun an unchanged behavior after a patch.

Understand the behavior before changing the example.

A small change needs two kinds of evidence



Read the diagram, then trace the same steps in code.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
def feature(inputs):	Give the change a boundary	Make behavior callable
[x for x in rows if ...]	Filter a collection	Choose the intended records
assert actual == expected	Compare against a prediction	Expose a wrong implementation
check.assertEqual(a, b)	Assert with unittest	Reuse checks in a test module

Python Basics · sections 07, 10, 15, 17 · use only when you need a lookup.

Make one predicate from one record

```python
def needs_work(row):
    return row["enabled"] and not row["finished"]

row = {"enabled": True, "finished": False, "published": False}
print(needs_work(row))
assert needs_work(row) is True
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Build a new result instead of changing input

```
01 numbers = [1, 2, 3]
02 selected = []
03 for number in numbers:
04     if number >= 2:
05         selected.append(number)
06 print(selected)
07 print(numbers)
08 assert numbers == [1, 2, 3]
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Keep an independent before snapshot

```
01 import copy
02 records = [{"points": 1}]
03 alias = records
04 before = copy.deepcopy(records)
05 records[0]["points"] = 9
06 print(alias[0]["points"], before[0]["points"])
07 assert before == [{"points": 1}]
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Check a boundary and filter a small collection

```
def at_least(value, minimum):
    return value >= minimum
for value in (4, 5, 6):
    print(value, at_least(value, 5))
records = [("A", False), ("B", True), ("C", False)]
pending = [name for name, finished in records if not finished]
print(pending)
```

1. Expected: 4 False; 5 True; 6 True; ['A', 'C'].
2. A predict decides which values to keep.
3. Replace >= with >: which check should fail?

Predict → execute → explain the value at each step.

Find a boundary bug in a function

```
01 def before_label(portions, reviewed):
02     if not reviewed:
03         return "hold"
04     return "team" if portions > 20 else "standard"
05
06 print(before_label(20, True))  # Wrong: standard.
07 print(before_label(20, False)) # Correct: hold.
```

1. Trace the early return first.
2. Compare > with >= at equality.
3. Keep an independent expected result.

An exact-boundary input can expose a one-character bug.

Table-driven checks for a small patch

```
def pickup_label(portions, reviewed):
    if not reviewed:
        return "hold"
    return "team" if portions >= 20 else "standard"

cases = [(19, True, "standard"), (20, True, "team"),
         (21, True, "team"), (20, False, "hold")]
for portions, reviewed, expected in cases:
    assert pickup_label(portions, reviewed) == expected
```

1. Each tuple supplies inputs and expectation.
2. The loop runs one check per tuple.
3. The first failing assert stops execution.

A table of cases checks the patch and preserved behavior.

List comprehensions: choose the right predicate

```
rows = [("D10", False, False),
        ("D11", True, False), ("D12", True, True)]
pending = [key for key, collected, delivered in rows
           if not collected]
wrong = [key for key, collected, delivered in rows
         if not delivered]
assert pending == ["D10"]
assert wrong == ["D10", "D11"]
print(pending, wrong)
```

1. Unpack each tuple into three names.
2. The if clause filters the result list.
3. Find the row that distinguishes the filters.

Changing a filter condition can change which items are included.

Import reusable code without running startup

```
01 from pickup_app import planned_portions
02
03 print(planned_portions(3, 3))
04 assert planned_portions(3, 3) == 9
05 # This import does not reserve or deliver anything.
```

1. Use the documented module location.
2. Tests can call a function directly.
3. Keep input/output work in the entry point.

Reusable code can be imported without starting the application.

unittest: assert a returned value

```
01 import unittest
02
03 def label(portions, reviewed):
04     if not reviewed:
05         return "hold"
06     return "team" if portions >= 20 else "standard"
07
08 check = unittest.TestCase()
09 check.assertEqual(label(20, True), "team")
10 check.assertEqual(label(20, False), "hold")
```

1. Import the actual function in your project tests
2. Retain independent expected labels
3. Add negative, failure and state-preservation cases

A test must execute an independent expectation, not remain skipped.

Hands-on practice

Week_09.ipynb · Session 2
Predict → run → change → explain

Your turn: practise the same Python skills

P1–P2 · Easy

Use the approach hints. Write the code yourself in a blank cell.

P3 · Core build

Implement a complete small program. Write expected results and assertions.

P4–P5 · Optional

Apply the skill to a new case, then attempt a harder challenge if time permits.

Write your own code, predict the result, then test and explain it.

Check your work before asking for help

A complete implementation

Write the whole operation. Do not stop at one changed character.

Independent expected results

Calculate a normal case and an edge case before running your program.

A peer can explain it

Swap driver and reviewer. Explain one line and one test.

Explain a filtering condition and test a small function against an expectation.

Project studio

Student-designed feature
Short design briefing → most time for your work

The people’s task this week

1

Someone needs help

Choose one small problem a volunteer actually faces.

2

This week, you will

Implement your approved small change and check it with another learner.

3

A useful result

A volunteer can try the changed feature and report what happened.

Use the supplied starting files. Focus on one useful result and check it.

Turn the need into an acceptance experiment

1

Need

Identify the coordinator decision and present difficulty.

2

Contract

Specify inputs, permitted action and observable output.

3

Cases

Predict a normal case and a distinguishing boundary or failure.

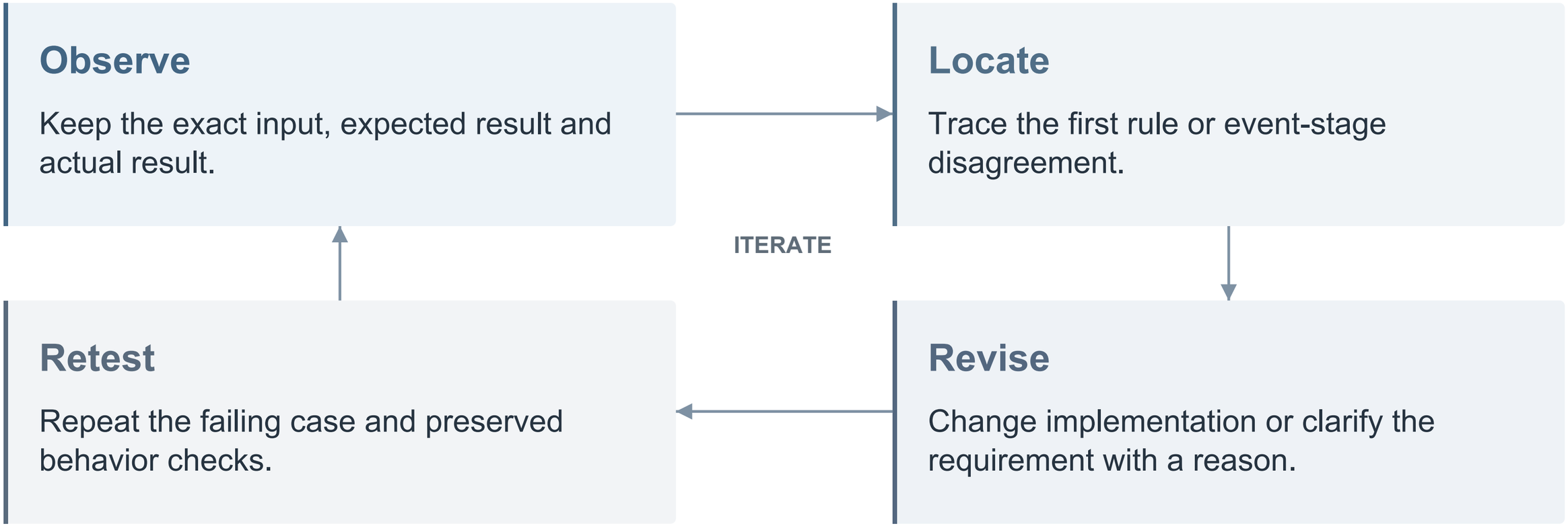
4

Evidence

Run independently and compare actual records and outputs.

Expected behavior comes from the need, not from the program output.

Use a failing observation to guide a revision



Find the first disagreement before changing the expected result.

Your implementation TODO

Supplied starting point

Baseline, fixtures and sample tests are supplied; they are the starting point, not your finished feature.

You implement / do

State the user need; implement the feature; write two acceptance cases plus an unaffected regression and a boundary/failure case.

Check the evidence

Run your actual feature entry point. Record a real peer finding, revise and retest. For a pending-collection view, distinguish collection from delivery.

Use the supplied baseline; explain a design decision and a limitation.

Show the component working

Step	What to show
1 · Locate	Identify the input, this component and its next user.
2 · Implement	Implement your approved feature and run actual tests.
3 · Check	Two acceptance cases; regression; failure; peer retest.
4 · Explain	Show your code, its actual result and why it behaves that way.

Use the supplied baseline; explain a design decision and a limitation.

Further reading in English

1. ## Python unittest documentation

   https://docs.python.org/3/library/unittest.html

2. ## Google Engineering Practices: How to do a code review

   https://google.github.io/eng-practices/review/reviewer/

Read one relevant section, then bring an example and a question.